Task 1 – Create and Write

Create a file named **notes.txt** and write three lines about Python using *write()*.

Program

```
# Task 1 - Create and Write
file = open("notes.txt", "w")
file.write("Python is a high-level programming language.\n")
file.write("It is easy to learn and widely used.\n")
file.write("Python is used in web development, AI and data science.\n")
file.close()
print("notes.txt created successfully")
```

Output

```
notes.txt created successfully
```

Explanation

open() in **w** mode creates notes.txt. Each *write()* adds one line, and **\n** moves to the next line.

Task 2 – Read the Whole File

Open **notes.txt** in read mode and display the entire contents using *read()*.

Program

```
# Task 2 - Read the Whole File
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

Output

```
Python is a high-level programming language.
It is easy to learn and widely used.
Python is used in web development, AI and data science.
```

Explanation

read() returns the whole file as a single string. The extra blank line at the end comes from the last **\n** in the file plus the newline added by *print()*.

Task 3 – Read One Line

Create a file named **students.txt** containing at least five student names. Use *readline()* to display only the first line.

Program

```
# Task 3 - Read One Line
with open("students.txt", "w") as file:
    file.write("Alekhya\n")
    file.write("Ravi\n")
    file.write("Sneha\n")
    file.write("Kiran\n")
    file.write("Priya\n")
    file.write("Arjun\n")

with open("students.txt", "r") as file:
    first_line = file.readline()
    print("First line:", first_line.strip())
```

Output

```
First line: Alekhya
```

Explanation

students.txt is created with six names. A single *readline()* reads only the first line, and *strip()* removes the newline character.

Task 4 – Read Line by Line

Using *readline()*, display the first three lines of **students.txt**, one line at a time.

Program

```
# Task 4 - Read Line by Line (first three lines of students.txt)
with open("students.txt", "r") as file:
    line1 = file.readline()
    line2 = file.readline()
    line3 = file.readline()

    print("Line 1:", line1.strip())
    print("Line 2:", line2.strip())
    print("Line 3:", line3.strip())
```

Output

```
Line 1: Alekhya
Line 2: Ravi
Line 3: Sneha
```

Explanation

Each call to *readline()* continues from where the previous call stopped, so three calls read the first three lines.

Task 5 – Read All Lines

Use *readlines()* to read all lines from **students.txt** and print the returned list.

Program

```
# Task 5 - Read All Lines
with open("students.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

Output

```
['Alekhya\n', 'Ravi\n', 'Sneha\n', 'Kiran\n', 'Priya\n', 'Arjun\n']
```

Explanation

readlines() returns a list in which every line is one string element, including its **\n** character.

Task 6 – Count Lines

Use *readlines()* to find and display the total number of lines in a file named **story.txt**.

Program

```
# Task 6 - Count Lines
# Create story.txt first so that the program can run on its own
with open("story.txt", "w") as file:
    file.write("Once upon a time, there was a small village.\n")
    file.write("A clever boy lived there with his grandmother.\n")
    file.write("One day he found a golden key in the forest.\n")
    file.write("The key opened a door to a hidden garden.\n")
    file.write("He shared the garden with the whole village.\n")

with open("story.txt", "r") as file:
    lines = file.readlines()
    print("Total number of lines:", len(lines))
```

Output

```
Total number of lines: 5
```

Explanation

readlines() gives a list of lines, so *len(lines)* is the total number of lines. The program first creates story.txt so that it can run on its own.

Task 7 – Write Multiple Lines

Use *write()* to store a student's name, age, course, and city on separate lines in **student_details.txt**.

Program

```
# Task 7 - Write Multiple Lines
with open("student_details.txt", "w") as file:
    file.write("Name: Alekhya\n")
    file.write("Age: 22\n")
    file.write("Course: Python Full Stack\n")
    file.write("City: Hyderabad\n")

print("Student details saved in student_details.txt")
```

Output

```
Student details saved in student_details.txt
```

Explanation

Four *write()* calls, each ending with **\n**, store the details on separate lines.

Task 8 – Append a New Record

Open **student_details.txt** in append mode and add a new student's details without deleting the existing content.

Program

```
# Task 8 - Append a New Record
with open("student_details.txt", "a") as file:
    file.write("\nName: Ravi\n")
    file.write("Age: 23\n")
    file.write("Course: Java Full Stack\n")
    file.write("City: Warangal\n")

print("New record added.\n")

with open("student_details.txt", "r") as file:
    print(file.read())
```

Output

```
New record added.

Name: Alekhya
Age: 22
Course: Python Full Stack
City: Hyderabad

Name: Ravi
Age: 23
Course: Java Full Stack
City: Warangal
```

Explanation

a mode adds new data at the end of the file, so the first student's details remain. The file is read again to verify both records.

Task 9 – Append Multiple Lines

Use *writelines()* to append three new lines to a file named **daily_log.txt**. Ensure each line ends with a newline character.

Program

```
# Task 9 - Append Multiple Lines
lines = [
    "Day 1: Learned Python basics\n",
    "Day 2: Practiced loops and functions\n",
    "Day 3: Completed file handling tasks\n"
]

with open("daily_log.txt", "a") as file:
    file.writelines(lines)

with open("daily_log.txt", "r") as file:
    print(file.read())
```

Output

```
Day 1: Learned Python basics
Day 2: Practiced loops and functions
Day 3: Completed file handling tasks
```

Explanation

writelines() does not add newlines by itself, so every string in the list ends with **\n**. If the program is run again, the same lines are appended again.

Task 10 – Write a List of Lines

Create **subjects.txt** and use *writelines()* to write a list of five subject names, each on a separate line.

Program

```
# Task 10 - Write a List of Lines
subjects = ["Python\n", "SQL\n", "HTML\n", "CSS\n", "JavaScript\n"]

with open("subjects.txt", "w") as file:
    file.writelines(subjects)

with open("subjects.txt", "r") as file:
    print(file.read())
```

Output

```
Python
SQL
HTML
CSS
JavaScript
```

Explanation

Each subject in the list ends with **\n**, so *writelines()* writes five separate lines.

Task 11 – Understand Write Mode

Create a file with some text, reopen it using **w** mode, write different text, and then read the file to observe what happened to the old content.

Program

```
# Task 11 - Understand Write Mode
# Step 1: create a file with some text
with open("sample.txt", "w") as file:
    file.write("This is the old text.\n")

with open("sample.txt", "r") as file:
    print("Before:")
    print(file.read())

# Step 2: reopen in 'w' mode and write different text
with open("sample.txt", "w") as file:
    file.write("This is the new text.\n")

# Step 3: read the file again
with open("sample.txt", "r") as file:
    print("After:")
    print(file.read())
```

Output

```
Before:
This is the old text.

After:
This is the new text.
```

Explanation

Observation: opening a file in **w** mode erases the old content. After writing new text, only the new text remains.

Task 12 – Understand Append Mode

Create a file with two lines, append two more lines using **a** mode, and verify that all four lines remain.

Program

```
# Task 12 - Understand Append Mode
# Step 1: create a file with two lines
with open("append_demo.txt", "w") as file:
    file.write("Line 1\n")
    file.write("Line 2\n")

# Step 2: append two more lines using 'a' mode
with open("append_demo.txt", "a") as file:
    file.write("Line 3\n")
    file.write("Line 4\n")

# Step 3: verify that all four lines remain
with open("append_demo.txt", "r") as file:
    print(file.read())
```

Output

```
Line 1
Line 2
Line 3
Line 4
```

Explanation

Observation: a mode keeps the old content and adds new lines at the end, so all four lines are present.

Task 13 – Exclusive Creation

Use **x** mode to create a new file named **unique.txt** and write a message into it. Run the program again and observe what happens if the file already exists.

Program

```
# Task 13 - Exclusive Creation
try:
    with open("unique.txt", "x") as file:
        file.write("This file was created using x mode.\n")
    print("unique.txt created successfully")
except FileExistsError:
    print("Error: unique.txt already exists. 'x' mode cannot overwrite it.")
```

Output

```
--- First run ---
unique.txt created successfully

--- Second run ---
Error: unique.txt already exists. 'x' mode cannot overwrite it.
```

Explanation

Observation: x mode creates a file only if it does not already exist. On the second run Python raises **FileExistsError**. Here it is handled with try/except; without it, the program would stop with an error.

Task 14 – Handle a Missing File

Try to open a file named **missing.txt** in **r** mode. Handle the situation gracefully if the file does not exist.

Program

```
# Task 14 - Handle a Missing File
try:
    with open("missing.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("Sorry, the file 'missing.txt' does not exist.")
```

Output

```
Sorry, the file 'missing.txt' does not exist.
```

Explanation

Opening a missing file in **r** mode raises **FileNotFoundError**. The try/except block catches it and prints a friendly message instead of crashing.

Task 15 – Copy File Contents

Read all contents from **source.txt** using *read()* and write them into a new file named **copy.txt**.

Program

```
# Task 15 - Copy File Contents
# Create source.txt first so that the program can run on its own
with open("source.txt", "w") as file:
    file.write("Python file handling is easy.\n")
    file.write("We can read, write and copy files.\n")
    file.write("This is the source file.\n")

with open("source.txt", "r") as source:
    data = source.read()

with open("copy.txt", "w") as copy:
    copy.write(data)

print("Contents copied to copy.txt\n")
with open("copy.txt", "r") as file:
    print(file.read())
```

Output

```
Contents copied to copy.txt

Python file handling is easy.
We can read, write and copy files.
This is the source file.
```

Explanation

The whole content of source.txt is read into one string and written to copy.txt. The program first creates source.txt so that it can run on its own.

Task 16 – Copy Lines Using readlines()

Read the lines from **source.txt** using *readlines()* and write them into **backup.txt** using *writelines()*.

Program

```
# Task 16 - Copy Lines Using readlines()
with open("source.txt", "r") as source:
    lines = source.readlines()

with open("backup.txt", "w") as backup:
    backup.writelines(lines)

print("Lines copied to backup.txt\n")
with open("backup.txt", "r") as file:
    print(file.read())
```

Output

```
Lines copied to backup.txt

Python file handling is easy.
We can read, write and copy files.
This is the source file.
```

Explanation

readlines() gives a list of lines, and *writelines()* writes that list into backup.txt. The newlines are already part of each line, so the format is preserved.

Task 17 – Search for a Word

Read a file named **article.txt** and check whether a word entered by the user appears in its contents.

Program

```
# Task 17 - Search for a Word
# Create article.txt first so that the program can run on its own
with open("article.txt", "w") as file:
    file.write("Python is a popular programming language.\n")
    file.write("It is used for web development, data science and automation.\n")

with open("article.txt", "r") as file:
    content = file.read()

word = input("Enter a word to search: ")
words_in_file = [w.strip(".,!?;:").lower() for w in content.split()]

if word.lower() in words_in_file:
    print(f"The word '{word}' is found in the file.")
else:
    print(f"The word '{word}' is not found in the file.")
```

Output

```
--- Run 1 ---
Enter a word to search: Python
The word 'Python' is found in the file.

--- Run 2 ---
Enter a word to search: Java
The word 'Java' is not found in the file.
```

Explanation

The file content is split into words (punctuation removed, lowercase). The word entered by the user is checked against that list, so the search is not case-sensitive. It is run twice here: once with a word that is present and once with a word that is not.

Task 18 – Count Words

Use *read()* to read **paragraph.txt** and display the number of words in the file.

Program

```
# Task 18 - Count Words
# Create paragraph.txt first so that the program can run on its own
with open("paragraph.txt", "w") as file:
    file.write("Python is a simple and powerful programming language. ")
    file.write("It is widely used in web development, data science and automation.")

with open("paragraph.txt", "r") as file:
    content = file.read()

word_count = len(content.split())
print("Number of words in the file:", word_count)
```

Output

```
Number of words in the file: 19
```

Explanation

read() returns the text, *split()* breaks it into words, and *len()* counts them.

Task 19 – Simple Notes Manager

Create a menu-driven program that lets the user write new notes, append a note, or display all notes. Use **w**, **a**, and **r** modes.

Program

```
# Task 19 - Simple Notes Manager (menu-driven)
filename = "my_notes.txt"

while True:
    print("\n===== NOTES MANAGER =====")
    print("1. Write a new note (replaces old notes)")
    print("2. Append a note")
    print("3. Display all notes")
    print("4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        note = input("Enter the note: ")
        with open(filename, "w") as file:       # w mode
            file.write(note + "\n")
        print("Note saved.")

    elif choice == "2":
        note = input("Enter the note to append: ")
        with open(filename, "a") as file:       # a mode
            file.write(note + "\n")
        print("Note appended.")

    elif choice == "3":
        try:
            with open(filename, "r") as file:   # r mode
                content = file.read()
            if content:
                print("--- Your Notes ---")
                print(content, end="")
            else:
                print("No notes found.")
        except FileNotFoundError:
            print("No notes found. Please write a note first.")

    elif choice == "4":
        print("Goodbye!")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output

```
===== NOTES MANAGER =====
1. Write a new note (replaces old notes)
2. Append a note
3. Display all notes
4. Exit
Enter your choice: 1
Enter the note: Learn Python daily
Note saved.

===== NOTES MANAGER =====
1. Write a new note (replaces old notes)
2. Append a note
3. Display all notes
4. Exit
Enter your choice: 2
Enter the note to append: Practice file handling
Note appended.

===== NOTES MANAGER =====
1. Write a new note (replaces old notes)
2. Append a note
3. Display all notes
4. Exit
Enter your choice: 3
--- Your Notes ---
Learn Python daily
Practice file handling

===== NOTES MANAGER =====
1. Write a new note (replaces old notes)
2. Append a note
3. Display all notes
4. Exit
Enter your choice: 4
Goodbye!
```

Explanation

Option 1 uses **w** mode (replaces old notes), option 2 uses **a** mode (adds a note), and option 3 uses **r** mode (shows all notes). The menu repeats until the user chooses Exit. Sample input used: 1, Learn Python daily, 2, Practice file handling, 3, 4.

Task 20 – Student Records File

Create a program that stores multiple student names and marks in **marks.txt** using *write()* or *writelines()*. Then read the file and display every record. Add one more record using append mode.

Program

```
# Task 20 - Student Records File
records = [
    "Alekhya,85\n",
    "Ravi,78\n",
    "Sneha,92\n",
    "Kiran,69\n"
]

# Store multiple records using writelines()
with open("marks.txt", "w") as file:
    file.writelines(records)

# Read the file and display every record
print("Student Records:")
with open("marks.txt", "r") as file:
    for line in file:
        name, marks = line.strip().split(",")
        print(f"Name: {name:<10} Marks: {marks}")

# Add one more record using append mode
with open("marks.txt", "a") as file:
    file.write("Priya,88\n")

print("\nAfter adding one more record:")
with open("marks.txt", "r") as file:
    for line in file:
        name, marks = line.strip().split(",")
        print(f"Name: {name:<10} Marks: {marks}")
```

Output

```
Student Records:
Name: Alekhya    Marks: 85
Name: Ravi       Marks: 78
Name: Sneha      Marks: 92
Name: Kiran      Marks: 69

After adding one more record:
Name: Alekhya    Marks: 85
Name: Ravi       Marks: 78
Name: Sneha      Marks: 92
Name: Kiran      Marks: 69
Name: Priya      Marks: 88
```

Explanation

Four records are stored using *writelines()* and displayed line by line. A fifth record is added in **a** mode, and the file is displayed again to show all five records.

Quick Revision

- **r** – opens an existing file for reading.
- **w** – opens a file for writing; creates it if needed and replaces existing content.
- **a** – opens a file for appending; new content is added at the end.
- **x** – creates a new file; raises FileExistsError if the file already exists.
- **read()** – reads the file contents.
- **readline()** – reads one line at a time.
- **readlines()** – returns the lines as a list.
- **write()** – writes a string to a file.
- **writelines()** – writes a sequence of strings; it does not automatically add newline characters.